Serverless Blue-Green Deployment Using AWS Cloud Services

B.Pardhuva
Department of CSE
IIIT Sri City
India
pardhuva.b23@iiits.in

C.Bhavishya
Department of CSE
IIIT Sri City
India
bhavishya.c23@iiits.in

G.Sai Vyshnavi
Department of CSE
IIIT Sri City
India
saivyshnavi.g23@iiits.in

***Abstract*—Modern cloud-native applications require deployment strategies that support frequent releases, high availability, and minimal operational overhead. Serverless computing on Amazon Web Services (AWS) addresses these requirements by abstracting away server management while providing automatic scaling and pay-per-use billing. In parallel, Blue-Green deployment strategies reduce downtime and deployment risk by running two identical environments and switching traffic between them. This paper presents the design and implementation of a serverless Blue-Green deployment architecture using AWS Lambda, Amazon API Gateway, Amazon Cognito, Amazon DynamoDB, and AWS Amplify. The proposed solution enables zero-downtime updates, safe and instant rollbacks, and controlled traffic shifting between Blue (stable) and Green (new) environments. Our implementation and evaluation show that the architecture effectively supports continuous delivery, reduces user impact during releases, and simplifies operational workflows.**

***Index Terms*—Serverless, Blue-Green Deployment, AWS Lambda, API Gateway, Cognito, DynamoDB, Amplify, Cloud Computing, CI/CD.**

I. Introduction

Cloud applications increasingly demand rapid iteration, continuous delivery, and resilient deployment mechanisms. Traditional deployment models based on virtual machines or on-premise servers often incur downtime during updates and require substantial operational effort for provisioning, scaling, and maintenance. These challenges make it difficult to release changes frequently without impacting end-users.

Serverless computing has emerged as a compelling paradigm in which the cloud provider manages infrastructure concerns such as capacity provisioning, patching, and auto-scaling. Developers focus primarily on writing business logic, while billing is based on actual execution time and resource consumption. AWS Lambda, combined with services such as Amazon API Gateway, Amazon DynamoDB, Amazon Cognito, and AWS Amplify, enables a fully managed, event-driven architecture.

Blue-Green deployment is a release strategy that reduces downtime and risk by maintaining two identical environments: Blue (the currently live environment) and Green (the new version). The new version is deployed to Green and tested thoroughly before production traffic is switched to it. If an issue is detected, traffic can be quickly routed back to Blue, enabling safe and fast rollback.

This work presents a serverless Blue-Green deployment model implemented on AWS. The architecture supports gradual traffic shifting using Lambda aliases, provides zero-downtime updates, and demonstrates how small, real-world applications can benefit from this approach.

II. Related Work

A variety of deployment patterns and tools have been proposed to improve the safety and efficiency of software releases.

A. Deployment Strategies

Common strategies include:

- **Recreate Deployment:** The existing version is stopped before the new version is deployed. This approach is simple but causes downtime.
- **Rolling Deployment:** Instances are updated gradually, with a subset of the environment updated at a time. This reduces downtime but complicates rollback.
- **Canary Deployment:** A small percentage of users receive the new version first, allowing monitoring under real traffic before full rollout.
- **Blue-Green Deployment:** Two complete environments run in parallel, and traffic is switched between them once the new version is validated.

Blue-Green deployment is particularly attractive when strict availability requirements exist, as the system can instantly fall back to the previous environment.

B. Cloud and Serverless Tooling

Cloud providers offer several services and frameworks for managing deployments:

- **AWS Elastic Beanstalk** supports environment cloning and URL swapping for Blue-Green deployment of web applications.
- **Amazon ECS and EKS** enable rolling and Blue-Green deployments for containerized workloads using load balancers and deployment controllers.
- **AWS CodeDeploy** supports Blue-Green deployments for EC2, Lambda, and ECS.

- **Serverless Framework, AWS SAM, and AWS CDK** simplify the definition and deployment of serverless applications.

While Blue-Green techniques are well-documented for container-based and server-based systems, there is comparatively less focus on practical, small-scale serverless Blue-Green deployments that integrate frontend, backend, authentication, and data storage. This paper addresses that gap by presenting a practical implementation on AWS.

III. Proposed Approach

The primary goal of the proposed approach is to combine the benefits of serverless computing with the reliability of Blue-Green deployment. The architecture is built around a fully managed, event-driven stack on AWS.

A. Motivation for Serverless

Serverless was chosen for the following reasons:

- **No server management:** AWS handles provisioning, scaling, and patching.
- **Scalability:** Automatic scaling with concurrent requests without manual configuration.
- **Cost efficiency:** Pay-per-use model is well-suited to workloads with variable or intermittent traffic.
- **Faster iteration:** Focus remains on business logic rather than infrastructure.

B. Motivation for Blue-Green Deployment

Blue-Green deployment complements serverless by:

- Minimizing downtime during releases.
- Reducing the impact of deployment errors on end-users.
- Enabling instant rollback if the new version misbehaves.
- Allowing real traffic testing on the Green environment before full cutover.

C. Architecture Overview

The solution uses the following AWS services:

- **AWS Lambda (Node.js):** Implements the backend logic for operations such as task management.
- **Amazon API Gateway:** Exposes RESTful APIs and routes requests to Lambda functions.
- **Amazon DynamoDB:** Stores structured application data, such as tasks and user-related records.
- **Amazon Cognito:** Manages user authentication and authorization.
- **AWS Amplify:** Hosts the React-based frontend and supports CI/CD for web deployments.

Two logical environments are maintained:

- **Blue Environment:** Stable version currently serving production traffic.
- **Green Environment:** Updated version deployed with UI changes and/or backend enhancements.

D. Traffic Management and Weighted Shifting

Traffic management is achieved using Lambda versions and aliases:

- Each deployment produces a new, immutable Lambda version.
- Aliases (e.g., `Blue`, `Green` or `prod`) point to specific versions.
- API Gateway is configured to invoke Lambda using an alias ARN.
- Weighted routing is applied at the alias level to gradually shift a small percentage of traffic (e.g., 10%) to the Green version, increasing it over time if no issues are observed.

If a problem is detected during the gradual rollout, the alias weights can be reverted to immediately restore all traffic to the Blue environment.

E. Implementation Process

The overall implementation process is summarized as:

1) **Version Creation:** Deploy backend changes as new Lambda versions and assign aliases for Blue and Green environments.
2) **Green Development:** Implement UI and functional updates, deploy them to the Green environment, and keep initial traffic at 0%.
3) **Testing and Validation:** Manually and automatically test the Green environment through direct API calls and frontend interaction.
4) **Traffic Migration:** Gradually shift traffic to Green using weighted aliases while monitoring errors and latency.
5) **Rollback Handling:** If any anomalies are detected, revert alias weights to route all traffic back to Blue.

IV. Experimental Setup

A. Application Stack

The prototype application is a task-oriented web system where users can add, view, and delete tasks via a React frontend. The stack is:

- **Frontend:** React application hosted on AWS Amplify, integrated with API Gateway and Cognito.
- **Backend:** AWS Lambda functions (Node.js) encapsulating business logic.
- **API Layer:** Amazon API Gateway providing RESTful endpoints.
- **Authentication:** Amazon Cognito user pools enforcing secure access.
- **Database:** Amazon DynamoDB storing task records and related data.

B. System Flow

The system flow is as follows:

1) The user interacts with the web application (e.g., adding or viewing a task).
2) The frontend sends an HTTP request to an API Gateway endpoint.

3) API Gateway forwards the request to the appropriate Lambda function based on the route.
4) The Lambda function executes the business logic, interacts with DynamoDB as needed, and returns a response.
5) API Gateway returns the processed result to the frontend, which updates the user interface.

C. Blue-Green Environment Setup

Two sets of Lambda versions and corresponding aliases were configured:

- **Blue:** Points to the stable Lambda version and the corresponding frontend build.
- **Green:** Points to the updated Lambda version and the new frontend build (e.g., UI enhancements).

Testing of the Green environment was performed using:

- Direct invocation of Green-specific endpoints.
- Interaction with the Green frontend build hosted in a separate Amplify environment.

D. Team Contributions

The implementation was carried out collaboratively:

- **Pardhuva:** Managed Lambda versions and aliases; configured Blue-Green environments; conducted testing, weighted traffic shifting, monitoring, and rollback experiments.
- **Bhavishya:** Implemented backend Lambda functions, designed DynamoDB data models, configured API Gateway endpoints, and worked with Amplify.
- **Sai Vyshnavi:** Focused on frontend development (UI and API integration) and integration of Cognito with the frontend.

V. Results

A. Functional Validation

The Green environment was validated to ensure that:

- Core functionalities such as add, view, and delete operations behaved as expected.
- UI updates rendered correctly across different browsers.
- Authentication and authorization flows remained intact under Cognito.

No functional regressions were observed when switching from Blue to Green.

B. Availability and Downtime

The Blue-Green deployment process, including weighted traffic shifting and full cutover, was executed without observed downtime:

- Users continued to interact with the application during deployments.
- Traffic cutover from Blue to Green was seamless from the user perspective.
- Rollback tests successfully redirected traffic from Green back to Blue without service interruption.

C. Operational Observations

Key operational observations include:

- The use of Lambda aliases greatly simplified traffic management and rollback.
- Monitoring through CloudWatch (errors, latency, and invocation metrics) provided clear visibility during traffic shifting.
- Maintaining separate but identical environments for Blue and Green improved confidence in production changes.

Although maintaining two environments can temporarily increase resource usage, the serverless pay-per-use model helps keep costs predictable, especially for small to medium workloads.

VI. Conclusion and Future Work

This paper presented a serverless Blue-Green deployment architecture using AWS cloud services. By combining AWS Lambda, Amazon API

VII. Conclusion and Future Work

This paper presented a serverless Blue-Green deployment architecture implemented using multiple AWS cloud services, including AWS Lambda, Amazon API Gateway, Amazon Cognito, Amazon DynamoDB, and AWS Amplify. By leveraging Lambda versions and aliases, as well as separate Blue and Green environments, the proposed approach enables:

- Zero-downtime deployments.
- Safe and immediate rollback in case of failures.
- Gradual and controlled traffic shifting using weighted routing.
- Reduced operational overhead through a fully managed serverless stack.

The experimental setup demonstrated that application functionality remained consistent during Blue-Green transitions and that user experience was not disrupted. The combination of serverless computing and Blue-Green deployment thus offers a practical and efficient solution for continuous delivery in modern cloud-native applications, especially for small to medium-scale systems.

A. Future Work

Future enhancements to this work may include:

- **Hybrid Canary + Blue-Green Deployments:** Integrating canary strategies with Blue-Green environments to further minimize the blast radius of faulty releases.
- **Advanced Monitoring and Automation:** Using CloudWatch alarms, Lambda-based automation, or third-party tools to automatically trigger rollbacks based on error thresholds and performance anomalies.
- **Multi-Region Deployments:** Extending the architecture to support multi-region Blue-Green deployments for global users, providing improved latency and disaster recovery capabilities.

- **Cost Optimization:** Investigating strategies to minimize the temporary cost overhead of maintaining parallel environments, especially for stateful or high-throughput workloads.

Overall, the project highlights that combining serverless architectures with robust deployment strategies such as Blue-Green can significantly improve the reliability and agility of cloud-based applications.

REFERENCES

[1] AWS Lambda Documentation. Available: https://aws.amazon.com/lambda/

[2] Amazon API Gateway Documentation. Available: https://aws.amazon.com/api-gateway/

[3] Amazon DynamoDB Documentation. Available: https://aws.amazon.com/dynamodb/

[4] AWS CodeDeploy Documentation. Available: https://aws.amazon.com/codedeploy/

[5] M. Fowler, "Blue-Green Deployment," Available: https://martinfowler.com/bliki/BlueGreenDeployment.html